

PROGRAMAÇÃO DE DISPOSITIVOS MÓVEIS

Prof. Tiago Piperno Bonetti
bonetti@prof.unipar.br



React Navigation

Documentação: <https://reactnavigation.org/docs/getting-started>

O React Navigation é composto de **alguns utilitários** principais e esses são usados pelos navegadores para criar a **estrutura de navegação** em seu aplicativo.

Precisamos envolver todo o aplicativo no *NavigationContainer*.

```
import 'react-native-gesture-handler';
import * as React from 'react';
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>{/* Rest of your app code */}</NavigationContainer>
  );
}
```

React Navigation

Em um navegador da web, você pode vincular a páginas diferentes usando um <a>.

Quando o usuário clica em um link, a URL é enviada para a pilha de histórico do navegador.

Quando o usuário pressiona o botão Voltar, o navegador exibe o item do topo da pilha do histórico.

O React Native não tem uma ideia incorporada de uma pilha de histórico global, é aqui que o React Navigation entra na história.

React Navigation

O React Navigation's **stack** fornece uma maneira para seu aplicativo fazer a transição entre telas e gerenciar o histórico de navegação.

Se seu aplicativo usa apenas um **stack navigator**, é conceitualmente semelhante a como um navegador da web lida com o estado de navegação.

Uma diferença importante é que o **stack navigator** do React Navigation fornece os gestos e animações que você esperaria no Android e iOS ao navegar entre as rotas na pilha.

Criando um stack navigator

createStackNavigator é uma função que retorna um objeto contendo 2 propriedades: *Screen* e *Navigator*.

Ambos são componentes React usados para configurar o navegador.

O ***Navigator*** deve conter elementos ***Screen*** como seus filhos para definir a configuração das rotas.

NavigationContainer é um componente que gerencia nossa árvore de navegação e contém o estado de navegação.

Este componente deve envolver toda a estrutura dos navegadores.

Normalmente, renderizamos esse componente na raiz do nosso aplicativo.

```
1  import * as React from 'react';
2  import { View, Text } from 'react-native';
3  import { NavigationContainer } from '@react-navigation/native';
4  import { createStackNavigator } from '@react-navigation/stack';
5
6  const TelaInicial = () => {
7    return (
8      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
9        <Text> Tela inicial </Text>
10      </View>
11    );
12  }
13
14  const Stack = createStackNavigator();
15
16  const App = () => {
17    return (
18      <NavigationContainer>
19        <Stack.Navigator>
20          <Stack.Screen name="Home" component={TelaInicial} />
21        </Stack.Navigator>
22      </NavigationContainer>
23    );
24  }
25
26  export default App;
```



❌ App.js (3:37) '@react-navigation/native' is not defined in dependencies. [Add dependency](#)

```

1  import * as React from 'react';
2  import { View, Text } from 'react-native';
3  import { NavigationContainer } from '@react-navigation/native';
4  import { createStackNavigator } from '@react-navigation/stack';
5
6  const TelaInicial = () => {
7    return (
8      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
9        <Text> Tela inicial </Text>
10     </View>
11   );
12 }
13
14 const TelaDetalhes = () => {
15   return (
16     <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
17       <Text>Tela de detalhes</Text>
18     </View>
19   );
20 }
21
22 const Stack = createStackNavigator();
23
24 const App = () => {
25   return (
26     <NavigationContainer>
27       <Stack.Navigator initialRouteName="TelaInicial">
28         <Stack.Screen name="Home" component={TelaInicial} />
29         <Stack.Screen name="Detalhes" component={TelaDetalhes} />
30       </Stack.Navigator>
31     </NavigationContainer>
32   );
33 }
34
35 export default App;

```

A **stack** tem duas rotas.

Uma rota pode ser especificada usando o componente **Screen**.

O componente **Screen** aceita uma propriedade **name** que corresponde ao nome da rota que usaremos para navegar e uma propriedade **component** que corresponde ao componente que ele renderizará.

A rota inicial para a pilha é a **TelaInicial**.

Altere a propriedade **initialRouteName** para ver a diferença

```

14  const Stack = createStackNavigator();
15
16  const App = () => {
17    return (
18      <NavigationContainer>
19        <Stack.Navigator initialRouteName="TelaInicial">
20          <Stack.Screen
21            name="Home"
22            component={TelaInicial}
23            options={{ title: 'Título da página' }}
24          />
25        </Stack.Navigator>
26      </NavigationContainer>
27    );
28  }
29
30  export default App;

```

Cada tela do navegador pode especificar algumas opções para o navegador, como o título a ser renderizado no cabeçalho.

Essas opções podem ser passadas na ***options*** para cada componente da tela.

Navegando para uma nova tela

```
5
6  const TelaInicial = ({ navigation }) => {
7    return (
8      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
9        <Text> Tela inicial </Text>
10       <Button
11         title="Ir para tela de detalhes"
12         onPress={() => navigation.navigate('Detalhes')}
13       />
14     </View>
15   );
16 }
```

A propriedade ***navigation*** é passada para cada **componente de tela** no ***stack navigator***.

Chamamos a função ***navigate*** com o nome da rota para a qual gostaríamos de mover o usuário.

navigate vs push

```
18  const TelaDetalhes = ({ navigation }) => {
19    return (
20      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
21        <Text>Tela de detalhes</Text>
22        <Button
23          title="Ir para tela de detalhes... Novamente"
24          onPress={() => navigation.navigate('Detalhes')}
25          //onPress={() => navigation.push('Detalhes')}
26        />
27      </View>
28    );
29  }
```

Cada vez que você chama **push**, adicionamos uma nova rota à pilha de navegação.

Ao chamar **navigate**, ele primeiro tenta encontrar uma rota existente com esse nome, e só envia uma nova rota se ainda não houver uma na pilha.

Voltando

```
18  const TelaDetalhes = ({ navigation }) => {
19    return (
20      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
21        <Text>Tela de detalhes</Text>
22        <Button
23          title="Ir para tela de detalhes... Novamente"
24          //onPress={() => navigation.navigate('Detalhes')}
25          onPress={() => navigation.push('Detalhes')}
26        />
27
28        <Button title="Ir para Home" onPress={() => navigation.navigate('Home')} />
29        <Button title="Voltar" onPress={() => navigation.goBack()} />
30      </View>
31    );
32  }
33 }
```

```

6  const TelaInicial = ({ navigation }) => {
7    return (
8      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
9        <Text> Tela inicial </Text>
10       <Button
11         title="Ir para tela de detalhes"
12         onPress={() => {
13           //Navegue até a rota Detalhes com os parâmetros
14           navigation.navigate('Detalhes', {
15             itemId: 10,
16             outroParametro: "o que você quiser"
17           });
18         }}
19       />
20     </View>
21   );
22 }
23
24 // adicionamos route
25 const TelaDetalhes = ({ route, navigation }) => {
26   //pegando os parâmetros
27   const { itemId } = route.params
28   const { outroParametro } = route.params
29   return (
30     <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
31       <Text>Tela de detalhes</Text>
32       //O método JSON.stringify() converte valores em javascript para uma String JSON.
33       <Text>itemId: {JSON.stringify(itemId)}</Text>
34       <Text>otherParam: {JSON.stringify(outroParametro)}</Text>
35       <Button
36         title="Ir para tela de detalhes... Novamente"
37         onPress={() => navigation.push('Detalhes')}
38       />
39       <Button title="Ir para Home" onPress={() => navigation.navigate('Home')} />
40       <Button title="Voltar" onPress={() => navigation.goBack()} />
41     </View>
42   );
43 }
44 }

```

Passando parâmetros para rotas

Passe parâmetros para uma rota, colocando-os em um objeto como um segundo parâmetro:

navigation.navigate('RouteName', { /* params go here */ })

Leia os parâmetros em seu componente de tela:

route.params.

```

6  const TelaInicial = ({ navigation, route }) => {
7    return (
8      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
9        <Button
10         title="Escrever um post"
11         onPress={() => navigation.navigate('CriarPost')}
12       />
13       <Text style={{ margin: 10 }}>Post: {route.params?.post}</Text>
14     </View>
15   );
16 }

17
18 const TelaCriarPost = ({ route, navigation }) => {
19   const [postTexto, setPostTexto] = React.useState('');
20   return (
21     <>
22       <TextInput
23         multiline
24         placeholder="No que você está pensando?"
25         style={{ height: 200, padding: 10, backgroundColor: 'white' }}
26         value={postTexto}
27         onChangeText={setPostTexto}
28       />
29       <Button
30         title="Criar"
31         onPress={() => {
32           // Pass and merge params back to home screen
33           navigation.navigate({
34             name: 'Home',
35             params: { post: postTexto },
36           });
37         }}
38       />
39     </>
40   );
41 }

```

Passando parâmetros para uma tela anterior

Exemplo: digamos que você tenha uma tela com um botão de criação de postagem, e o botão de criação de postagem abre uma nova tela para criar uma postagem. Depois de criar a postagem, você deseja passar os dados da postagem de volta para a tela anterior.

Para fazer isso, você pode usar o método ***navigate***, que funciona como ***goBack*** se a tela já existisse.

Você pode passar o ***params*** com ***navigate*** para devolver os dados.

O que deve estar nos parâmetros

Params são como opções para uma tela.

Eles devem conter apenas informações para configurar o que é exibido na tela.

Evite passar todos os dados que serão exibidos na própria tela (por exemplo, passe um id de usuário em vez de um objeto de usuário).

Além disso, evite transmitir dados que são usados por várias telas, esses dados devem estar em um armazenamento global.

Você também pode pensar no objeto de rota como um URL.

Se sua tela tivesse um URL, o que deveria estar no URL?

O que deve estar nos parâmetros

Params não devem conter dados que você acha que não deveriam estar no URL.

Isso geralmente significa que você deve manter o mínimo possível de dados necessários para determinar o que é a tela.

Pense em visitar um site de compras, ao ver listas de produtos, o URL geralmente contém o nome da categoria, tipo de classificação, quaisquer filtros, etc., não contém a lista real de produtos exibidos na tela.

O que deve estar nos parâmetros

Exemplo: digamos que você tenha uma tela **Profile**.

Ao navegar até ela, você pode ficar tentado a passar o objeto do usuário nos parâmetros:

```
// Don't do this
navigation.navigate('Profile', {
  user: {
    id: 'jane',
    firstName: 'Jane',
    lastName: 'Done',
    age: 25,
  },
});
```

Dados como objetos de usuário devem estar em seu armazenamento global em vez de no estado de navegação. Caso contrário, você terá os mesmos dados duplicados em vários lugares.

A melhor maneira é passar apenas o ID do usuário nos parâmetros:

```
navigation.navigate('Profile', { userId: 'jane' });
```


O que deve estar nos parâmetros

Alguns exemplos do que deveria estar nos parâmetros são:

- 1) IDs como ID de usuário, ID de item etc.
- 2) Parâmetros para classificação, filtragem de dados etc. quando você tem uma lista de itens, por exemplo ***navigation.navigate('Feeds', { sortBy: 'latest' })***
- 3) Números de página ou cursores para paginação, por exemplo ***navigation.navigate('Chat', { beforeTime: 1603897152675 })***
- 4) Dados para preencher entradas em uma tela para compor algo, por exemplo ***navigation.navigate('ComposeTweet', { title: 'Hello world!' })***

Em essência, passe a menor quantidade de dados necessária para identificar uma tela em parâmetros.

Para muitos casos, isso significa simplesmente passar o ID de um objeto em vez de passar um objeto completo.

Referências

Escudelario, Bruna de Freitas; Pinho, Diego Martins. **React Native: Desenvolvimento de aplicativos mobile com React**. Casa do código.

Documentação React Native. Disponível em: <https://reactnative.dev/docs>.

Documentação React Navigation. Disponível em: <https://reactnavigation.org/docs/getting-started>